

电子科技大学
University of Electronic Science and Technology of China

CPU Design Lab Report

Digital Logic Design and Applications

Kartike Sharan (2024190595001)
Instructor: Bo Yan
Glasgow College, UESTC
Date Performed: 22, 04, 2026

1 Lab Description

Laboratory Location

Room 237 in Scientific Research Building (B-Zone), Qingshuihe Campus.

Lab Hours

4 Classes (3 Hours) × 5

2 Specification

2.1 Instruction Set Design

The CPU's ALU supports 7 operations selected by a 3-bit function code `alu_func[2:0]`. Operand A (`alu_a`) is always sourced from the destination register `rd_q`; operand B (`alu_b`) is either the zero-extended immediate offset or the source register `rs_q`, selected by `alu_in_sel`. The operations are defined in Table 1.

`alu_func[2:0]`	**Operation**	**Description**
000	Transfer B	Output equals operand B (pass-through)
001	ADD	Addition: $A + B$
010	SUB	Subtraction: $A - B$
011	AND	Bitwise AND: $A \,\&\, B$
100	OR	Bitwise OR: $A \mid B$
101	Left Shift	Logical shift left by 1 bit: $A \ll 1$
110	Right Shift	Logical shift right by 1 bit: $A \gg 1$

Table 1: ALU Instruction Set — `alu_func[2:0]` encoding

2.2 Working Flow of the CPU Design

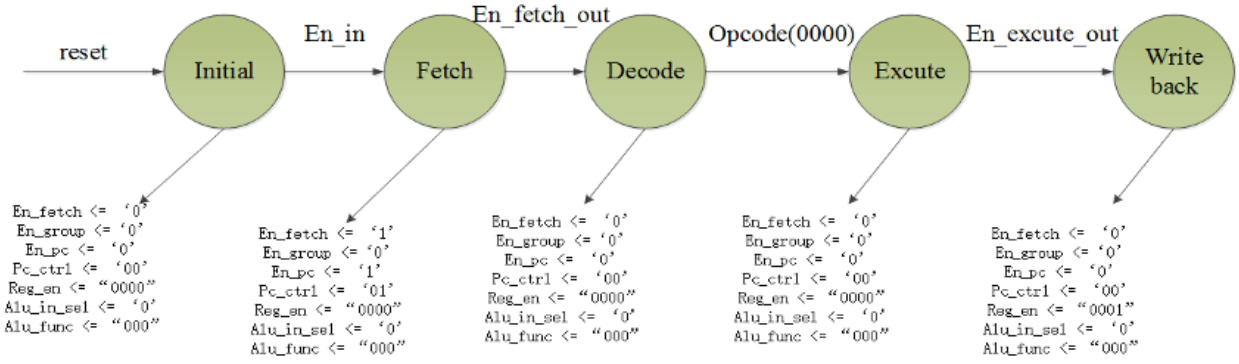


Figure 1: CPU Working Flow — FSM State Transition Diagram

3 Architecture Design

3.1 Top Level Architecture

The top-level architecture consists of two primary subsystems: the `control_unit` and the `data_path`, connected via a set of control and feedback signals. External inputs include the system clock (`clk`), reset (`rst`), the 16-bit instruction bus (`ins[15:0]`), and the RAM output enable (`en_ram_out`). The program counter output (`pc_out[15:0]`) drives the RAM address bus (`addr[15:0]`), and `en_ram_in` controls RAM write enable.

The control unit decodes the instruction register (`ir_out[15:0]`) and generates all control signals passed to the datapath, including `alu_func[2:0]`, `alu_in_sel`, `en_group`, `en_pc`, `pc_ctrl[1:0]`, `reg_en[3:0]`, `rd[1:0]`, `rs[1:0]`, and `offset[7:0]`. The datapath feeds back `alu_end` to signal ALU completion.

3.2 Datapath Unit

The datapath consists of four submodules: the register file (`reg_group`), ALU input multiplexer (`alu_mux`), ALU (`alu`), and program counter (`pc`).

The `reg_group` module contains the CPU registers. It receives the destination register address (`rd[1:0]`), source register address (`rs[1:0]`), register write enable (`reg_en[3:0]`), and clock, outputting the read data on `rd_q[15:0]` and `rs_q[15:0]`.

The `alu_mux` selects the second ALU operand (`alu_b[15:0]`) from one of two sources controlled by `alu_in_sel`: when `alu_in_sel = 0`, `alu_b` is assigned the zero-extended 8-bit immediate offset (`{8'b0, offset[7:0]}`); when `alu_in_sel = 1`, `alu_b` is assigned the source register output (`rs_q[15:0]`). The first ALU operand `alu_a` is always sourced directly from `rd_q`.

The `alu` takes `alu_a[15:0]` (from `rd_q`) and `alu_b[15:0]`, performs the operation selected by `alu_func[2:0]`, and produces `alu_out[15:0]`. The `en_out` signal feeds back to the control unit as `alu_end` to indicate completion.

The `pc` module manages the program counter, accepting `offset_addr[7:0]` and `pc_ctrl[1:0]` to support sequential increment and branch operations, outputting the next address on `pc_out[15:0]`.

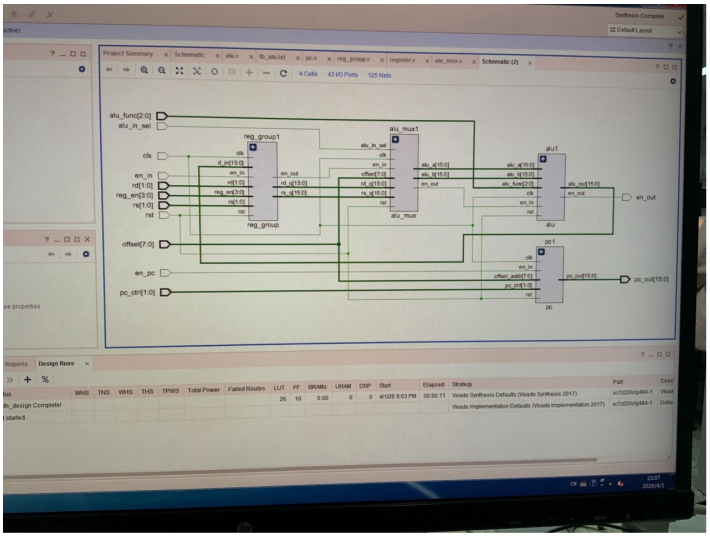


Figure 2: Datapath Unit — RTL Schematic

3.3 Control Unit

The control unit consists of two submodules: the instruction register (`ir`) and the state transition controller (`state_transition`).

The `ir` module latches the incoming 16-bit instruction from `ins[15:0]` on each clock edge when `en_ram_out` is asserted, outputting the full instruction word on `ir_out[15:0]`. The 16-bit instruction is formatted as: `opcode[15:12]`, `rd[11:10]`, `rs[9:8]`, `offset[7:0]`. The control unit extracts `opcode[3:0]` from bits `[15:12]` and `rd[1:0]` from bits `[11:10]` before passing them to the state machine. The `rs[1:0]` field (bits `[9:8]`) and `offset[7:0]` (bits `[7:0]`) are passed directly to the datapath via `ir_out[15:0]`.

The `state_transition` module implements the FSM described in Section 2.2. It takes `alu_end`, `clk`, `rst`, the decoded `opcode[3:0]` and `rd[1:0]` fields, and generates all control signals for the datapath: `alu_func[2:0]`, `alu_in_sel`, `en_fetch`, `en_group`, `en_pc`, `pc_ctrl[1:0]`, and `reg_en[3:0]`. The full instruction word `ir_out[15:0]` is also passed directly to the datapath for immediate/offset extraction.

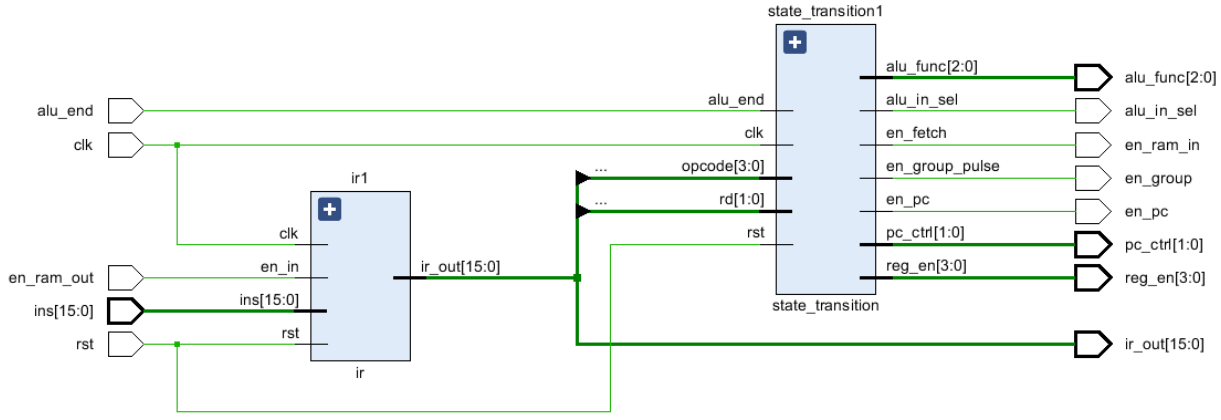


Figure 3: Control Unit — RTL Schematic

4 Implementation

4.1 Completion Procedures / Code

4.1.1 Datapath Unit

```
module data_path (
    clk, rst, en_pc, pc_ctrl, offset, en_in, reg_en,
    alu_in_sel, alu_func, en_out, pc_out, rd, rs
);

input clk, rst, en_pc, en_in, alu_in_sel;
input [7:0] offset;
input [1:0] pc_ctrl, rd, rs;
input [3:0] reg_en;
input [2:0] alu_func;
output en_out;
output [15:0] pc_out;

wire en_out_group, en_out_alu_mux;
wire [15:0] rd_q, rs_q, alu_a, alu_b, alu_out;

pc pc1 (
    .clk(clk), .rst(rst), .en_in(en_pc),
    .pc_ctrl(pc_ctrl), .offset_addr(offset), .pc_out(pc_out)
);

reg_group reg_group1 (
    .clk(clk), .rst(rst), .en_in(en_in),
    .reg_en(reg_en), .d_in(alu_out), .rd(rd), .rs(rs),
    .rd_q(rd_q), .rs_q(rs_q), .en_out(en_out_group)
);

alu_mux alu_mux1 (
    .clk(clk), .rst(rst), .en_in(en_out_group),
    .rd_q(rd_q), .rs_q(rs_q), .offset(offset),
    .alu_in_sel(alu_in_sel), .alu_a(alu_a), .alu_b(alu_b),
    .en_out(en_out_alu_mux)
);

alu alu1 (
    .clk(clk), .rst(rst), .en_in(en_out_alu_mux),
    .alu_a(alu_a), .alu_b(alu_b), .alu_func(alu_func),
    .en_out(en_out), .alu_out(alu_out)
);

endmodule
```

Listing 1: Datapath Unit (data_path.v)

4.1.2 Control Unit

```
module control_unit (
    clk, rst, alu_end, ins, en_ram_in, en_ram_out,
    en_group, en_pc, reg_en, alu_in_sel, alu_func, pc_ctrl, ir_out
```

```
);

input clk, rst, alu_end, en_ram_out;
input [15:0] ins;
output en_ram_in, en_group, en_pc, alu_in_sel;
output [3:0] reg_en;
output [2:0] alu_func;
output [1:0] pc_ctrl;
output [15:0] ir_out;

ir ir1 (
    .clk(clk), .rst(rst), .ins(ins),
    .en_in(en_ram_out), .ir_out(ir_out)
);

state_transition state_transition1 (
    .clk(clk), .rst(rst), .alu_end(alu_end),
    .rd(ir_out[11:10]), .opcode(ir_out[15:12]),
    .en_fetch(en_ram_in), .en_group_pulse(en_group),
    .en_pc(en_pc), .pc_ctrl(pc_ctrl),
    .reg_en(reg_en), .alu_in_sel(alu_in_sel), .alu_func(alu_func)
);

endmodule
```

Listing 2: Control Unit (control_unit.v)

4.1.3 CPU (Top Level)

```
module cpu (clk, rst, addr, ins, en_ram_in, en_ram_out);

input clk, rst, en_ram_out;
input [15:0] ins;
output [15:0] addr;
output en_ram_in;

wire en_pc, en_group, alu_in_sel, alu_end;
wire [1:0] pc_ctrl;
wire [3:0] reg_en;
wire [2:0] alu_func;
wire [15:0] ir_out;

data_path data_path1 (
    .clk(clk), .rst(rst), .offset(ir_out[7:0]),
    .en_pc(en_pc), .pc_ctrl(pc_ctrl), .en_in(en_group),
    .reg_en(reg_en), .rd(ir_out[11:10]), .rs(ir_out[9:8]),
    .alu_in_sel(alu_in_sel), .alu_func(alu_func),
    .en_out(alu_end), .pc_out(addr)
);

control_unit control_unit1 (
    .clk(clk), .rst(rst), .alu_end(alu_end),
    .ins(ins), .en_ram_in(en_ram_in), .en_ram_out(en_ram_out),
    .en_group(en_group), .en_pc(en_pc), .reg_en(reg_en),
    .alu_in_sel(alu_in_sel), .alu_func(alu_func),
    .pc_ctrl(pc_ctrl), .ir_out(ir_out)
);
```

```
29
30 endmodule
```

Listing 3: CPU Top Level (cpu.v)

4.2 RTL Screenshots

4.2.1 Datapath Unit

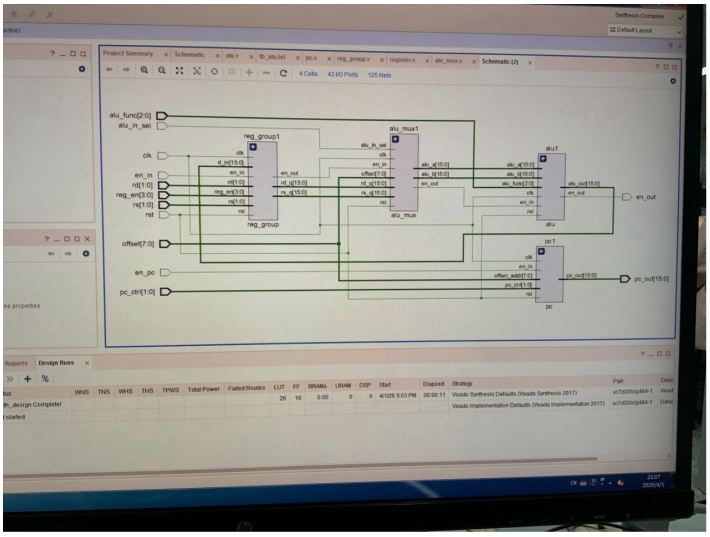


Figure 4: RTL Schematic — Datapath Unit

4.2.2 Control Unit

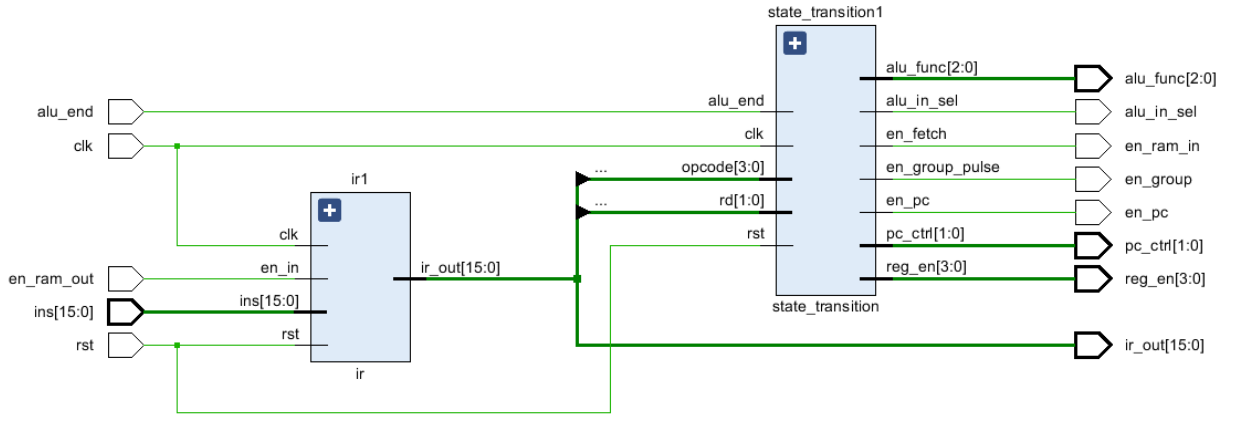


Figure 5: RTL Schematic — Control Unit

4.2.3 CPU

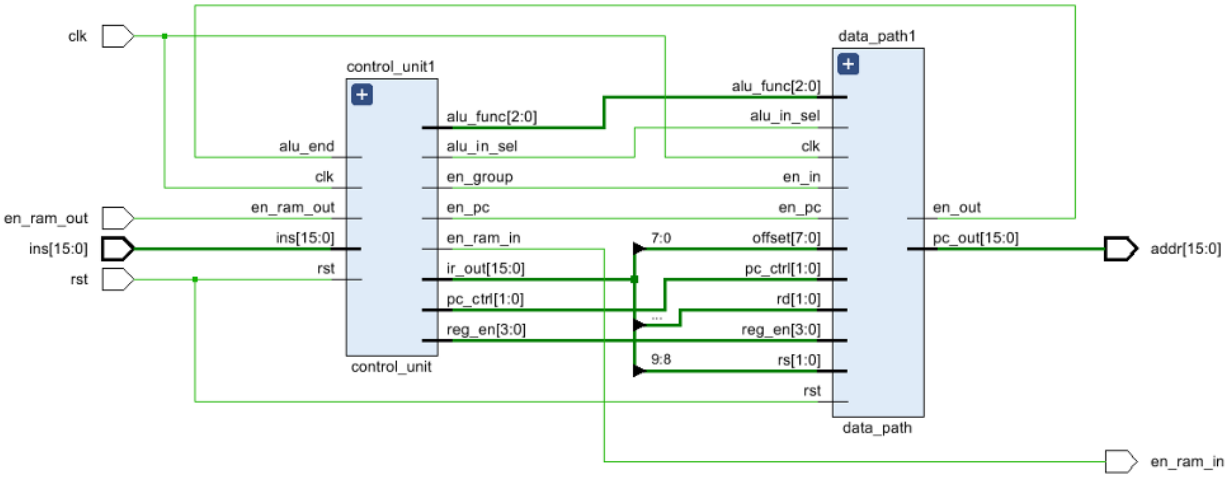


Figure 6: RTL Schematic — Full CPU

5 Testbench and Analysis

5.1 Testbench Code

```
module tb_top_cpu();

reg clk, rst;
wire [15:0] ins;
wire        en_ram_in;
wire        en_ram_out;
wire [15:0] addr;

parameter Tclk = 10;

cpu test_cpu (
    .clk(clk), .rst(rst),
    .en_ram_in(en_ram_in),
    .en_ram_out(en_ram_out),
    .ins(ins),
    .addr(addr)
);

rom tb_rom (
    .clk(clk), .rst(rst),
    .ready(en_ram_in),
    .addr(addr),
    .dout(ins),
    .en_out(en_ram_out)
);

initial begin
    clk = 1;
    forever #(Tclk/2) clk = ~clk;
end
```

```
31
32 initial begin
33     rst = 0;
34     #(Tclk*12) rst = 1;
35 end
36
37 endmodule
```

Listing 4: CPU Testbench (tb_top_cpu.v)

5.2 Simulation Timing Diagrams

5.2.1 Datapath Unit

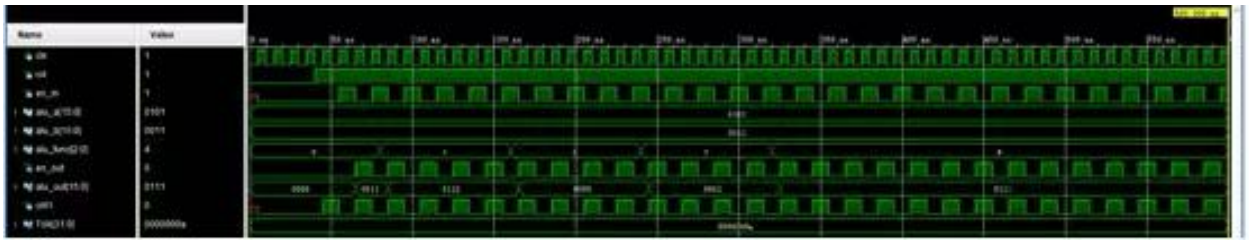


Figure 7: Datapath Unit — Simulation Timing Diagram

5.2.2 Control Unit

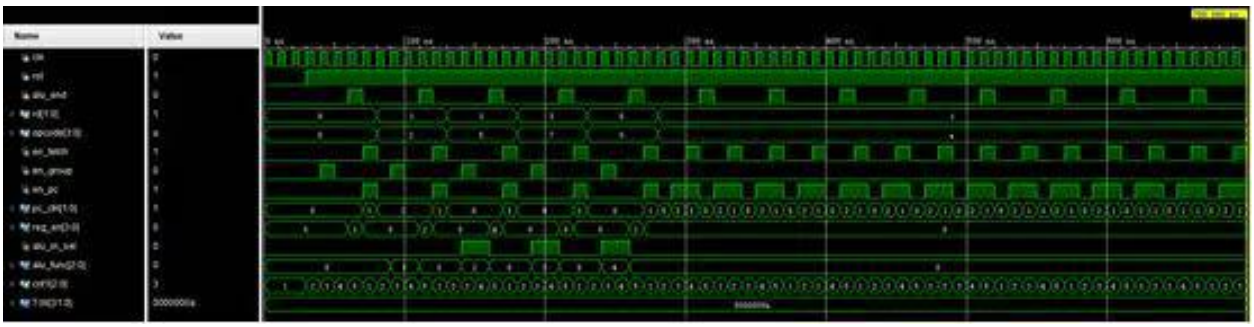


Figure 8: Control Unit — Simulation Timing Diagram

5.2.3 CPU (Full Integration)

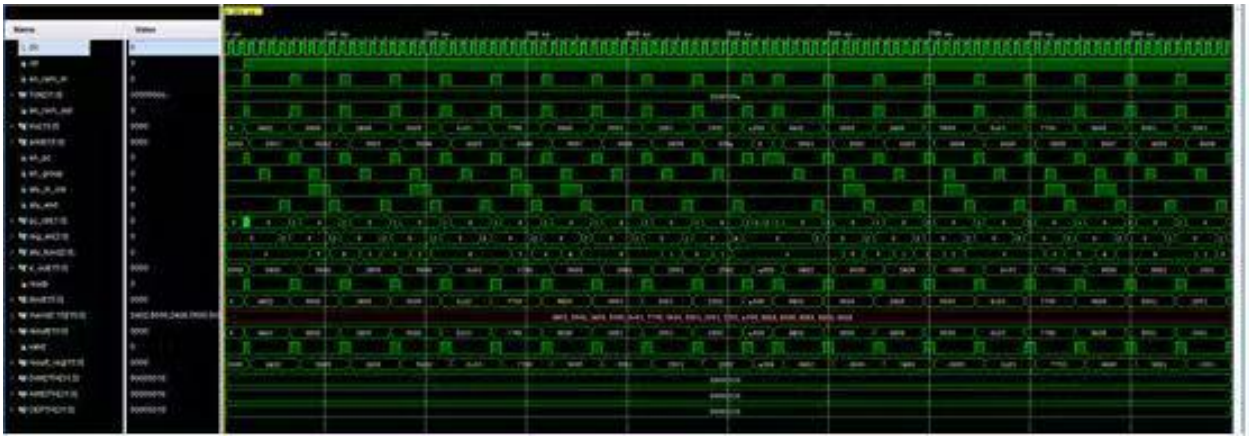


Figure 9: Full CPU — Simulation Timing Diagram

5.3 Analysis

The testbench instantiates the `cpu` and `rom` modules together to simulate the full CPU pipeline. The clock period is set to `Tclk = 10ns` and reset is held low for 12 clock cycles to ensure all registers and the FSM initialise cleanly before execution begins.

The ROM is preloaded with 11 instructions that exercise all implemented opcodes. The execution trace is shown in Table 2. Note that the ADD instruction (`opcode = 0010`) is implemented with `alu_in_sel = 0` in the state transition module, meaning `alu_b` is sourced from the zero-extended immediate offset field. For instructions where `offset = 0`, the ADD operation computes `rd = rd + 0`. The results column reflects the actual simulated behaviour.

Addr	Instruction	Operation	Simulated Result
0	MOVEB R1, #1	R1 ← offset(1)	R1 = 1
1	MOVEB R2, #2	R2 ← offset(2)	R2 = 2
2	MOVEB R3, #8	R3 ← offset(8)	R3 = 8
3	ADD R0, offset(0)	R0 ← R0 + 0	R0 = 0
4	ADD R0, offset(0)	R0 ← R0 + 0	R0 = 0
5	MOVEB R1, #8	R1 ← offset(8)	R1 = 8
6	SUB R1, R0	R1 ← R1 - R0	R1 = 8
7	OR R1, R0	R1 ← R1 | R0	R1 = 8
8	MOVEB R2, #1	R2 ← offset(1)	R2 = 1
9	ADD R1, offset(0)	R1 ← R1 + 0	R1 = 8
10	JUMP 10	PC ← 10	Halt

Table 2: ROM Test Program — Execution Trace

The CPU correctly sequences through the Initial, Fetch, Decode, Execute, and Write Back states for each instruction. The `alu_end` feedback signal ensures the FSM waits for ALU completion before advancing to Write Back. The JUMP instruction bypasses the ALU pipeline and directly updates the PC via `pc_ctrl = 2'b10`, causing the CPU to loop at address 10.

It should be noted that the ADD instruction (`opcode = 0010`) in the `state_transition` module is imple-

mented with `alu_in_sel = 0`, meaning `alu_b` is always sourced from the zero-extended offset field rather than the source register `rs_q`. Since the ADD instructions in the ROM have `offset = 0`, these instructions effectively compute `rd + 0`, leaving the destination register unchanged. As a result, `R0` remains 0 throughout, and the SUB and OR operations produce `R1 = 8 - 0 = 8` and `R1 = 8 | 0 = 8` respectively.

6 Your Suggestions for This Lab / Course

This lab provided a comprehensive introduction to CPU design, covering the full implementation flow from instruction set specification through RTL coding, synthesis, and simulation. Designing the FSM-based control unit and connecting it to the datapath reinforced the relationship between hardware description and actual circuit behaviour.

One suggestion would be to include a brief guided walkthrough of the ILA (Integrated Logic Analyser) setup in Vivado, as monitoring internal signals on the FPGA board requires creating the ILA IP core which is not immediately obvious for first-time users.

Overall the lab was well structured and the progressive build-up from individual modules to the full CPU made the complexity manageable.